Universidade de Brasília — CIC0236 Teleinformática e Redes 2 — 2026/2

RDT-UnB: janela deslizante com Go-Back-N e Selective Repeat — Relatório da Entrega 2

Grupo `grupo07`: Lucas Adeodato de Oliveira (231013645), Lurian Correia Lima (222005395), Atila Sousa Fernandes (231013510)

1 Introdução

Na Entrega 1 mostramos que o Stop-and-Wait (SAW) é limitado pelo RTT: com um único segmento em trânsito, o cliente passa mais de 99% do tempo esperando. Nesta entrega implementamos dois protocolos de janela deslizante (*pipelining*) sobre o mesmo protocolo RDT-UnB v1.0 e o mesmo servidor *pull-based*: o Go-Back-N (GBN) e o Selective Repeat (SR) [1]. Nos dois, o cliente mantém até N `REQ`s enviados sem `DATA` de volta; eles diferem no tratamento de perdas e de segmentos fora de ordem. Comparamos SAW, GBN e SR com janelas de 4, 8 e 16, sem perda e com 5% e 10% de perda simulada.

2 Implementação

O cliente ganhou as opções `-protocol {saw,gbn,sr}`, `-window N`, `-loss P` e `-seed S`. Handshake (`HELLO`/`OK`), encerramento (`FIN`/`DONE` com MD5) e o CSV de métricas são os mesmos da Entrega 1. Só o laço de transferência muda (`src/saw.py`, `src/gbn.py` e `src/sr.py`). Como o servidor só envia `DATA` quando recebe `REQ`, a "janela"é a de pedidos: o cliente faz o papel de emissor (decide o que pedir e quando reenviar) e também de receptor (aceita, descarta ou guarda os `DATA`). O timeout é fixo, como no SAW, e o `ACK|seq=n` é enviado assim que o `DATA` n é aceito.

2.1 Go-Back-N

O cliente guarda `base` (segmento mais antigo sem `DATA`) e `next` (próximo a pedir) e mantém a janela cheia, com $\texttt{next} - \texttt{base} \leq N$. O receptor só aceita o `DATA` da `base`: qualquer outro (fora de ordem, atrasado ou duplicado) é descartado. Há um único temporizador, o da `base`, contado a partir do último `REQ` dela. Quando ele vence, o cliente reenvia **todos** os `REQ`s de `base` até `next`-1 e soma 1 à coluna `retransmissions` de cada um (Figura 1).

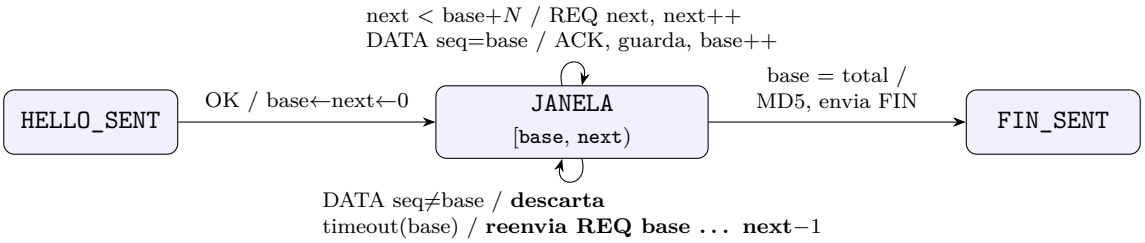


Figura 1: Máquina de estados do cliente GBN (rótulos *evento / ação*). Esgotadas 10 retransmissões da mesma base, a sessão termina com erro.

2.2 Selective Repeat

O SR mantém a mesma janela de pedidos, mas cada segmento pendente tem o **próprio** temporizador. Um `DATA` de qualquer segmento pendente é aceito, confirmado com `ACK` e guardado num *buffer* indexado por `seq`, mesmo que chegue fora de ordem. A `base` desliza por cima de todos os segmentos consecutivos já presentes no *buffer*, liberando espaço na janela para novos pedidos. Quando o temporizador de um segmento vence, **só** o `REQ` daquele segmento é reenviado. `DATA` de segmento

que já não está pendente (duplicata) é descartado. Ao final, o arquivo é remontado concatenando o *buffer* em ordem de `seq` (Figura 2).

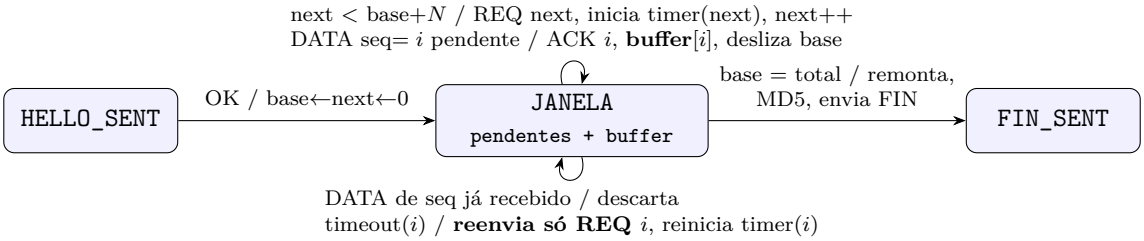


Figura 2: Máquina de estados do cliente SR. Cada segmento pendente i tem seu próprio temporizador; até 10 retransmissões por segmento.

2.3 Simulação de perda

Escolhemos a abordagem (a): **descartar o `DATA` recebido com probabilidade** P antes de processá-lo. O pacote chega do socket, mas o cliente sorteia um número com `random.Random(seed)` (opção `-seed`, padrão 42). Se o número for menor que P (`-loss`), o `DATA` é ignorado como se tivesse se perdido na rede, e o segmento continua pendente até o timeout. Escolhemos descartar o `DATA`, e não o `ACK`, porque no RDT-UnB quem dispara a retransmissão é a falta do `DATA`: o servidor é *pull-based* e não reenvia nada por conta própria, e o próprio `ACK` não é processado pelo servidor, que responde `ERROR|msg=tipo_desconhecido_ACK` a cada um. Assim, um `ACK` omitido não teria efeito nenhum sobre a transferência, enquanto um `DATA` descartado exercita exatamente o caminho de perda, timeout e retransmissão de cada protocolo. A semente fixa torna os cenários reproduzíveis.

No CSV, a coluna `loss_simulated` (o campo *simulated_loss* do enunciado) vale 1 se pelo menos um `DATA` daquele segmento foi descartado pela simulação e 0 caso contrário. Isso separa a perda simulada das retransmissões causadas por perda natural da rede: um segmento com `retransmissions`>0 e `loss_simulated`$=0$ foi reenviado por perda real ou, no GBN, por ter sido arrastado pela retransmissão da janela. Na demonstração ao vivo, o cliente roda com `-loss 0` (padrão).

3 Metodologia

Os 21 cenários (SAW; GBN e SR com $N = 4$, 8 e 16; perdas de 0, 5 e 10%) foram executados contra o servidor `137.131.178.229:8080` em 26/10/2026, entre 15:00 e 15:42, com o arquivo `small` (2 048 segmentos de 512 B), timeout fixo de 1 000 ms e semente 42. O script `scripts/entrega02_executa.py` roda os cenários em ordem intercalada (para cada perda, todos os protocolos), para que uma piora momentânea da rede não afete só um protocolo. O script `scripts/entrega02_analisa.py` gera as tabelas e os gráficos a partir dos CSVs. O throughput médio é bytes recebidos $\times$ 8/tempo total, com o tempo medido do primeiro `REQ` até o último `DATA`. As retransmissões são a soma da coluna `retransmissions`.

4 Resultados

4.1 Integridade

Protocolo	Janela	Sem perda	5%	10%
SAW	1	completo* (2048 seg.)	completo* (2048 seg.)	MD5 ok, `DONE ok=1`
GBN	4	completo* (2048 seg.)	completo* (2048 seg.)	MD5 ok, `DONE ok=1`
GBN	8	completo* (2048 seg.)	completo* (2048 seg.)	MD5 ok, `DONE ok=1`
GBN	16	completo* (2048 seg.)	completo* (2048 seg.)	MD5 ok, `DONE ok=1`
SR	4	completo* (2048 seg.)	MD5 ok, `DONE ok=1`	MD5 ok, `DONE ok=1`
SR	8	completo* (2048 seg.)	MD5 ok, `DONE ok=1`	MD5 ok, `DONE ok=1`
SR	16	completo* (2048 seg.)	MD5 ok, `DONE ok=1`	MD5 ok, `DONE ok=1`

Tabela 1: Integridade dos arquivos recebidos. Todas as execuções com log preservado produziram o MD5 `317d6890df9867ccda199afe6a362aba`, igual ao `checksum` informado no `OK`. *Em 11 cenários a saída do cliente (com o MD5) foi sobrescrita por uma reexecução posterior; o CSV registra o download completo de todos os segmentos.

4.2 Tabela comparativa

		Sem perda			5% de perda			10% de perda		
Protocolo	Janela	kbit/s	Retx.	Tempo (s)	kbit/s	Retx.	Tempo (s)	kbit/s	Retx.	Tempo (s)
SAW	1	181,9	0	46,1	38,9	137	215,8	27,4	220	306,3
GBN	4	529,4	0	15,8	52,1	408	161,1	27,4	1 010	306,4
GBN	8	723,6	0	11,6	53,6	848	156,6	31,8	1 912	263,5
GBN	16	1 226,2	16	6,8	72,4	1 592	115,9	33,4	3 878	251,5
SR	4	374,1	3	22,4	83,4	91	100,6	36,3	250	231,2
SR	8	1 058,4	0	7,9	98,1	91	85,5	55,1	221	152,2
SR	16	2 132,2	0	3,9	136,0	91	61,7	77,0	234	109,0

Tabela 2: Throughput médio (bytes recebidos ×8 / tempo total), total de retransmissões de `REQ` e tempo total da transferência do arquivo `small` em cada cenário.

4.3 Gráficos

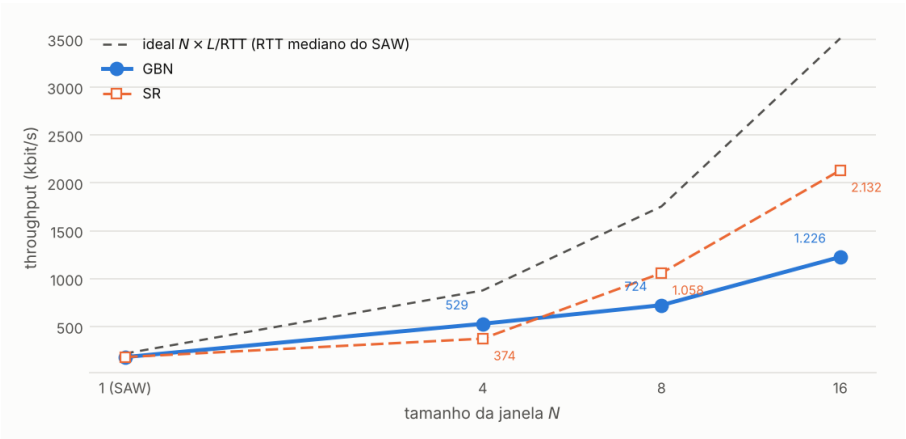


Figura 3: Throughput médio sem perda em função do tamanho da janela, comparado ao ideal $N \times L/\mathrm{RTT}$. Sem perda, as curvas de GBN e SR praticamente se sobrepõem.

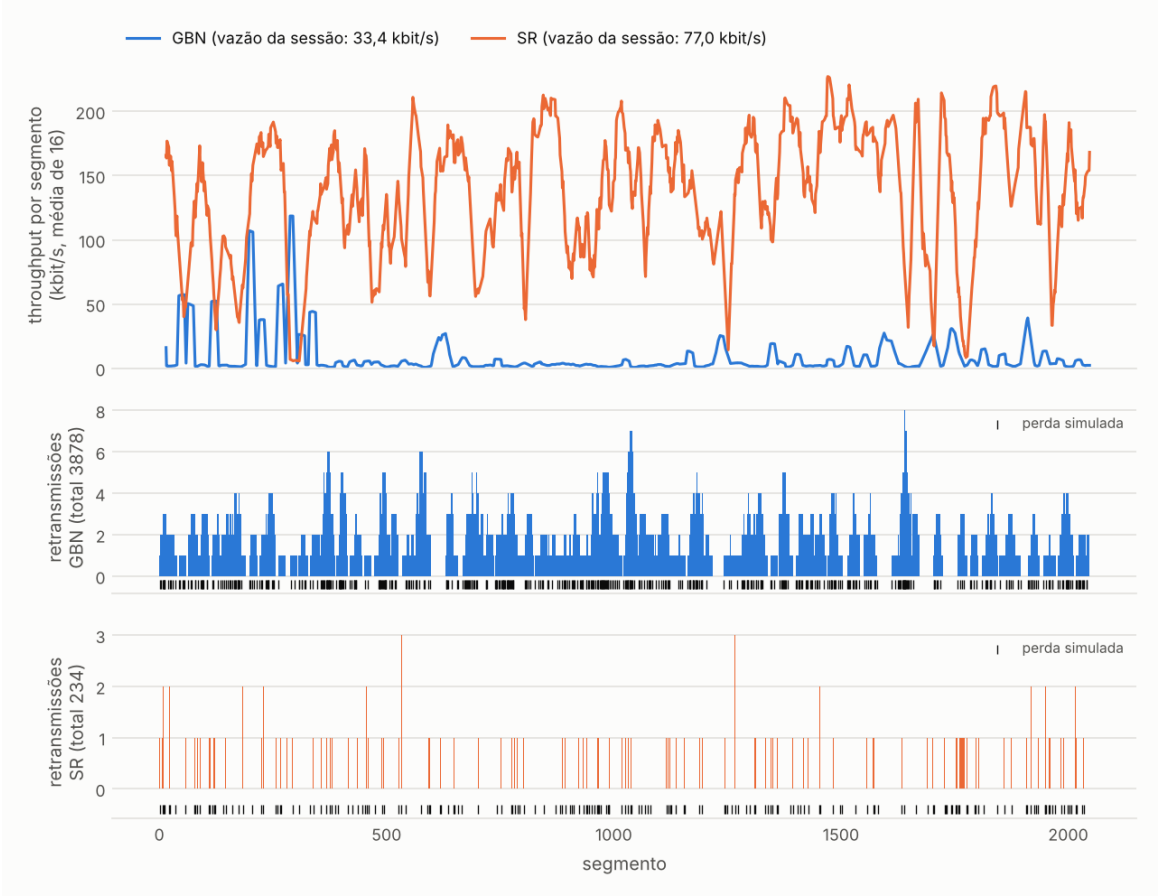


Figura 4: GBN e SR com janela 16 e 10% de perda. Topo: throughput por segmento (coluna `throughput_kbps`, média móvel de 16 segmentos); um segmento retransmitido inclui o timeout no seu tempo. Abaixo: retransmissões por segmento. Os traços marcam os segmentos com perda simulada.

5 Análise

(a) Impacto do tamanho da janela sem perda

No GBN, o throughput passou de 181,9 kbit/s (SAW, $N = 1$) para 529,4, 723,6 e 1 226,2 kbit/s com $N = 4$, 8 e 16 (2,9×, 4,0× e 6,7× o SAW). De 4 para 8 o ganho foi de 37% e de 8 para 16, de 69%. No SR, o throughput passou de 181,9 kbit/s (SAW, $N = 1$) para 374,1, 1 058,4 e 2 132,2 kbit/s com $N = 4$, 8 e 16 (2,1×, 5,8× e 11,7× o SAW). De 4 para 8 o ganho foi de 183% e de 8 para 16, de 101%.

Sem perda, GBN e SR executam o mesmo caminho (pedem, recebem em ordem e confirmam, sem retransmissões), então diferenças entre eles nessa coluna vêm das condições da rede em cada execução, e não do protocolo. Com N segmentos em trânsito, chegam até N segmentos por RTT, e o teto ideal é $N \times L/\text{RTT}$ (com RTT mediano de 18,68 ms, 877, 1 755 e 3 509 kbit/s para $N = 4$, 8 e 16). Esse crescimento linear vale enquanto a janela é menor que o produto banda × atraso do caminho. Quando $N \times L$ passa a cobrir todo o RTT, o canal fica sempre ocupado e o limite passa a ser o gargalo (a banda do enlace ou a taxa de resposta do servidor e do cliente), não mais a janela.

Nos dados, o throughput **não se estabilizou até** $N = 16$: cada vez que a janela dobrou, a vazão ainda subiu de forma expressiva, sinal de que o produto banda × atraso do caminho não foi atingido. A estabilização só ocorreria com janelas maiores. A maior vazão medida (2 132,2 kbit/s) equivale a $\approx 9,7$ segmentos entregues por RTT mediano, abaixo dos 16 da janela, porque parte do tempo a janela fica parada esperando o `DATA` mais lento. A estabilização ocorreria na janela em que $N \times L/\text{RTT}$ alcançasse a banda disponível. Ver Figura 3.

Dois cuidados na leitura desses números. Primeiro, cada cenário foi executado uma vez, e o RTT

mediano variou de 18,7 a 38,0 ms entre as execuções. Medida contra o ideal $N \times L/\text{RTT}$ da própria execução, a eficiência ficou entre 55% e 83%, porque os picos de RTT seguram a janela e o `DATA` mais lento atrasa o deslizamento. Segundo, mesmo sem perda simulada houve **perda natural** na rede (retransmissões com `loss_simulated`= 0): no GBN com $N = 16$, 1 timeout (16 `REQ`s reenviados) custou ≈ 1 s de 6,8 s; sem ele, a vazão seria ≈ 1 436 kbit/s; no SR com $N = 4$, 3 timeouts (3 `REQ`s reenviados) custaram ≈ 3 s de 22,4 s; sem eles, a vazão seria ≈ 432 kbit/s. Numa transferência de poucos segundos, um único timeout de 1 s pesa muito na média.

(b) Retransmissões por erro com 10% de perda e janela 16

No GBN, cada timeout da base reenvia a janela inteira, somando +1 na coluna `retransmissions` de até 16 segmentos consecutivos. Separando esses blocos no CSV, houve 243 timeouts e 3878 `REQ`s reenviados: **16,0 segmentos retransmitidos por erro**, praticamente a janela inteira. No SR, cada timeout reenvia um único `REQ`: houve 234 retransmissões para 193 segmentos com `loss_simulated`= 1, ou **1,21 por erro**. O valor fica um pouco acima de 1 porque o próprio reenvio também pode ser descartado. No total, o GBN reenviou **16,6**× mais `REQ`s que o SR. O GBN também marcou mais segmentos com perda simulada (469 contra 193), porque recebe muito mais `DATA`s, e cada reenvio da janela é exposto de novo aos 10% de descarte (Figura 4).

Um exemplo do CSV: no GBN, o primeiro timeout ocorreu na base `seq=1`, e as linhas 1 a 16 (o bloco de 16 segmentos daquela janela) passaram a ter `retransmissions`≥ 1, todos reenviados por causa de um único `DATA` perdido. No SR, o primeiro segmento retransmitido foi o `seq=1`, e nenhum dos três vizinhos de cada lado foi reenviado.

(c) Por que o GBN sofre mais que o SR com alta perda

Com 10% de perda (Tabela 2), com $N = 4$, o GBN fez 1 010 retransmissões e levou 306,4 s (27,4 kbit/s), contra 250 retransmissões e 231,2 s (36,3 kbit/s) do SR; com $N = 8$, o GBN fez 1 912 retransmissões e levou 263,5 s (31,8 kbit/s), contra 221 retransmissões e 152,2 s (55,1 kbit/s) do SR; com $N = 16$, o GBN fez 3 878 retransmissões e levou 251,5 s (33,4 kbit/s), contra 234 retransmissões e 109,0 s (77,0 kbit/s) do SR.

Com 10% de perda, aumentar a janela de 4 para 16 mudou o throughput do GBN em +22% e o do SR em +112%.

A diferença vem do receptor. O GBN não guarda segmentos fora de ordem: quando o `DATA` da base se perde, os $N - 1$ `DATA`s seguintes, que chegaram corretamente, são descartados, e todos são pedidos de novo após o timeout. Com perda p, a chance de uma janela de N segmentos passar sem nenhum erro é $(1 - p)^N$, que vale 19% para $p = 10\%$ e $N = 16$. Quase toda janela tem um erro, e cada erro custa até N reenvios. Quanto maior a janela, mais trabalho já feito é jogado fora a cada erro. O SR guarda os segmentos fora de ordem no buffer e reenvia só o que faltou: o custo de um erro é um `REQ`, independentemente de N. Enquanto espera o timeout do segmento perdido, ele continua recebendo o resto da janela, e por isso janelas maiores continuam ajudando o SR mesmo com perda.

6 Conclusão

Os 21 cenários terminaram com o arquivo completo, e em todas as 10 execuções com log preservado, de SAW, GBN e SR, o MD5 conferiu e o servidor respondeu `DONE|ok=1`. Isso confirma a remontagem correta mesmo com segmentos fora de ordem no SR. Sem perda, a janela deslizante eliminou o limite de um segmento por RTT do SAW: com $N = 16$, GBN e SR chegaram a 1 226,2 e 2 132,2 kbit/s, 6,7× e 11,7× o SAW. A diferença entre os dois sem perda veio das condições da rede em cada execução (perda natural e variação do RTT), e não do protocolo. Com 10% de perda e $N = 16$, o GBN fez 3 878 retransmissões contra 234 do SR, e o SR teve a maior vazão (77,0 contra 33,4 kbit/s): cada erro custa até N reenvios no GBN e um único reenvio no SR. Em todos os protocolos, o timeout

fixo de 1 000 ms ainda é o maior custo de cada perda, dezenas de vezes o RTT. Isso motiva o timeout adaptativo por EWMA da próxima entrega.

7 Referências bibliográficas

[1] J. F. Kurose e K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken: Pearson, 2021.

[2] J. Postel, "User Datagram Protocol," RFC 768, IETF, 1980. Disponível em: https://www.rfc-editor.org/rfc/rfc768.

[3] R. Rivest, "The MD5 Message-Digest Algorithm," RFC 1321, IETF, 1992. Disponível em: https://www.rfc-editor.org/rfc/rfc1321.